



***Creation of Workforce Insights Dashboard
for Employee Skill and Analytics***

Complete Project Documentation

1. Project Overview

The Workforce Insights Dashboard for Employee Skill and Analytics is an end-to-end HR analytics system that predicts employee attrition risk, identifies skill gaps, generates AI-driven retention and training recommendations, and presents all of this through an interactive web dashboard. The system also includes a Retrieval-Augmented Generation (RAG) chatbot that answers workforce-policy questions grounded in the organization's own knowledge base.

1.1 End-to-End Project Flow



1.2 Technology Stack Summary

Layer	Technology
Data Cleaning & EDA	Python, Pandas, Jupyter Notebook
Feature Engineering	Pandas, NumPy, custom transformation pipeline
Database	PostgreSQL 15, hosted on Supabase (Session Pooler, SSL)
Machine Learning	Scikit-learn (Random Forest), LightGBM, Joblib
Recommendation Engine	Python rule-based logic → JSON output
RAG / AI Assistant	FAISS + TF - IDF, Groq cloud API
Backend API	FastAPI, SQLAlchemy + pg8000, Pydantic
Frontend / Dashboard	FastAPI + Jinja2 templates, HTML5, CSS3, Chart.js

2. Data Cleaning and Transformation

2.1 Overview

This section summarizes the data cleaning and quality-assurance process applied to the employee attrition dataset. The workflow covers structural inspection, missing-value and duplicate checks, standardization of column names and text fields, logical/business-rule validation, outlier detection using the IQR method, and a final encoding step that converts binary categorical fields into numeric flags before exporting the cleaned dataset.

Item	Detail
Raw records	15,000 rows
Raw columns	32 columns
Final records	15,000 rows (no rows removed)
Final columns	32 columns (5 columns re-typed to numeric)
Missing values found	0 across all columns
Duplicate rows found	0
Output file	employee_attrition_cleaned_dataset.csv

2.2 Data Loading

The dataset was loaded directly from the provided CSV export using pandas (`df = pd.read_csv("employee_attrition_dataset.csv")`). An initial preview and shape check confirmed 15,000 rows and 32 columns, matching the expected size of the raw dataset.

2.3 Structural Standardization

2.3.1 Column Name Normalization

Column headers were originally in mixed/Title case (e.g., `Employee_ID`, `Years_At_Company`). All column names were converted to lowercase and stripped of leading/trailing whitespace, e.g. `Employee_ID` → `employee_id`, `Years_At_Company` → `years_at_company`.

2.3.2 Data Type Audit

`df.info()` and `df.dtypes` were used to confirm each column's data type. The dataset contains a mix of numeric fields (age, monthly_income, salary_hike_percent, job_satisfaction, distance_from_home) and categorical/text fields (gender, marital_status, education_level, department, job_role, employment_type, overtime, remote_work, business_travel, promotion_last_5_years, stock_option, attrition). All numeric fields already carried appropriate types, so no type coercion was needed.

2.4 Missing Values and Duplicate Records

- Missing values: every one of the 32 columns returned a count of 0 missing values (0.0%). No imputation, row removal, or placeholder-filling was required.
- Duplicate records: `df.duplicated().sum()` returned 0. The primary key `employee_id` was also verified unique (0 duplicates).

2.5 Categorical & Text Field Cleaning

Each object-type column was profiled with `value_counts()` to inspect category labels and catch inconsistent casing, typos, or unexpected categories. All 12 categorical columns returned clean, consistent category sets, for example:

- gender: Female, Male
- marital_status: Divorced, Married, Single
- education_level: Bachelor, High School, Master, PhD
- business_travel: Frequently, Never, Rarely
- overtime / remote_work / promotion_last_5_years / stock_option / attrition: No, Yes

Two further checks confirmed the text data needed no additional cleaning: a whitespace check (0 mismatches against stripped values across every column) and an empty-string check (0 blank strings hiding as valid entries in any column).

2.6 Logical Consistency Checks

Beyond structural cleanliness, the data was validated against real-world business logic. In every case the filtered result was an empty DataFrame — no violations were found.

Rule	Check	Result
Role tenure cannot exceed company tenure	<code>years_in_current_role > years_at_company</code>	0 violations
Company tenure cannot exceed working-age years	<code>years_at_company > (age - 18)</code>	0 violations
Income must be positive	<code>monthly_income <= 0</code>	0 violations
Project count cannot be negative	<code>number_of_projects < 0</code>	0 violations

2.7 Outlier Detection (IQR Method)

Numeric columns were first visualized with box plots to get a quick sense of spread. Formal outlier detection was then performed using the interquartile range (IQR) method: $Q1$, $Q3$, $IQR = Q3 - Q1$, lower = $Q1 - 1.5 \times IQR$, upper = $Q3 + 1.5 \times IQR$. Values outside this range were flagged as outliers but not removed — they were reviewed and retained as plausible real-world data points, consistent with treating this as a business dataset rather than a purely statistical sample.

2.7.1 Monthly Income — Detailed Review

Lower limit: 727.5 | Upper limit: 16,059.5. 320 employees fell above the upper bound (high earners, largely IT Managers, Software Engineers, and Financial Analysts). These records were reviewed alongside `employee_id`, `job_role`, and `department`, and judged to be legitimate senior/specialist salaries rather than data errors, so they were kept.

2.7.2 Outlier Scan Across All Numeric Columns

Column	Outliers
years_at_company	702
years_in_current_role	898
monthly_income	320
salary_hike_percent	78
job_level	733
performance_rating	595
environment_satisfaction	1,067
relationship_satisfaction	872
training_hours_last_year	47
distance_from_home	439
commute_time_minutes	536
number_of_projects	308
team_size	10
manager_rating	853
absence_days	313
age / job_satisfaction / work_life_balance / attrition_risk_score / employee_id	0

Given the volume and business plausibility of these flagged values (tenure and satisfaction scores naturally cluster and produce many statistical 'outliers' in IQR terms even though they are valid), no rows were dropped or capped. The flags were treated as informational for downstream analysis rather than errors to correct.

2.8 Binary Field Encoding

As a final transformation, five Yes/No categorical columns were converted to numeric binary flags (1 = Yes, 0 = No) so the dataset is model-ready: attrition, stock_option, promotion_last_5_years, remote_work, overtime.

Column	Before	After
attrition	No / Yes (object)	0 / 1 (int64)
stock_option	No / Yes (object)	0 / 1 (int64)
promotion_last_5_years	No / Yes (object)	0 / 1 (int64)
remote_work	No / Yes (object)	0 / 1 (int64)
overtime	No / Yes (object)	0 / 1 (int64)

Remaining categorical fields (gender, marital_status, education_level, department, job_role, employment_type, business_travel) were intentionally left as text, since they are multi-category and are better suited to one-hot/label encoding at the modeling stage.

2.9 Export

The cleaned DataFrame was written to disk as employee_attrition_cleaned_dataset.csv, containing all 15,000 original records and 32 columns, with standardized lowercase column names and five columns re-typed from Yes/No text to 0/1 integers.

2.10 Summary of Actions Taken

- Standardized all column headers to lowercase and stripped whitespace.
- Verified there were no missing values in any of the 32 columns.
- Verified there were no duplicate rows and no duplicate employee_id values.
- Profiled all categorical columns and confirmed clean, consistent category labels.
- Validated four business-logic rules — zero violations found.
- Ran IQR-based outlier detection across all numeric columns and reviewed high-income outliers individually; retained all rows as legitimate data.
- Converted five Yes/No columns into numeric 0/1 flags.
- Exported the final cleaned dataset (15,000 rows × 32 columns).

3. Feature Engineering

3.1 What is Feature Engineering?

Feature engineering means creating useful new columns from existing employee information. Instead of using only the original values, existing fields are combined or transformed to answer HR questions more easily.

Before	After	Why it helps
Years at company + years in role	Role tenure ratio	Shows how much of an employee's company time is spent in the current role.
4 satisfaction scores	Overall satisfaction index	Gives one simple measure of overall employee satisfaction.
Projects + team size	Workload intensity	Gives an indication of workload relative to team size.
Performance + manager rating	Performance-manager gap	Shows differences between performance and manager assessment.
Several warning indicators	HR red-flag count	Summarizes employee risk signals.

3.2 Dataset Before and After

Dataset	Rows	Columns	Meaning
Cleaned Dataset	15,000	32	Clean and usable employee data
Featured Engineering Dataset	15,000	63	Cleaned data + useful derived features

3.3 Steps Followed

Step	What was done	Simple explanation
Step 1	Load cleaned dataset	Read the cleaned employee CSV.
Step 2	Check the data	Check rows, columns, missing values and duplicates.
Step 3	Create career features	Add indicators for role movement, tenure and career stagnation.
Step 4	Create salary features	Compare income with job level and department.
Step 5	Create satisfaction features	Combine satisfaction measures.
Step 6	Create workload features	Use projects and team size to estimate workload.
Step 7	Create commute features	Identify commute burden and long commute.
Step 8	Create performance features	Compare performance and manager ratings.
Step	What was done	Simple explanation

Step 9	Create attendance/risk features	Use absence, overtime and satisfaction as HR signals.
Step 10	Encode categories	Convert text categories into numerical columns.
Step 11	Validate and export	Check the final dataset and save the CSV.

3.4 Main New Features — Simple Explanation

Feature	What it means	Why useful
is_senior_age	Identifies employees aged 50 or above.	Helps study workforce age patterns.
role_tenure_ratio	Current-role years compared with company years.	Shows how long employees remain in the same role.
has_changed_role	Whether current-role tenure is less than company tenure.	Helps study internal movement.
career_stagnation_flag	Flags long role tenure with no recent promotion.	Can indicate possible career stagnation.
years_per_job_level	Company tenure relative to job level.	Helps understand progression.
income_per_job_level	Income relative to job level.	Makes salary comparisons easier.
income_percentile_in_dept	Employee's income position within the department.	Shows relative pay.
below_dept_median_income_flag	Flags income below department median.	Can support compensation analysis.
salary_hike_per_year_tenure	Salary hike relative to tenure.	Shows salary growth context.
overall_satisfaction_index	Average of four satisfaction measures.	Provides one overall satisfaction value.
satisfaction_volatility	Variation among satisfaction measures.	Shows whether satisfaction is consistent.
low_satisfaction_flag	Flags relatively low overall satisfaction.	Useful for employee-risk analysis.
workload_intensity	Projects relative to team size.	Shows workload pressure.
projects_per_year_tenure	Projects relative to company tenure.	Normalizes project exposure.
business_travel_ordinal	Travel category converted to numbers.	Makes travel information usable in models.
commute_burden_index	Commute time relative to home distance.	Adds commute context.
Feature	What it means	Why useful

long_commute_flag	Flags long commute.	Useful for wellbeing analysis.
performance_manager_gap	Performance rating minus manager rating.	Shows rating differences.
high_performer_flag	Flags high performers.	Useful for talent management.
underrated_flag	Performance is higher than manager rating.	Can highlight recognition gaps.
absence_rate_per_year	Absence normalized by tenure.	Makes absence easier to compare.
high_absence_flag	Flags high absence.	Potential HR warning signal.
overtime_and_low_satisfaction_flag	Flags overtime plus low satisfaction.	A combined warning signal.
hr_red_flag_count	Counts selected HR warning indicators.	Provides a simple employee risk count.
burnout_risk_score	Combines several observable warning indicators.	Provides a simple burnout-risk measure.

3.5 Categorical Encoding

Computers and machine-learning models work with numbers, so text categories were converted into numerical columns.

Original information	Example encoded columns	Simple meaning
Education	education_level_ordinal	High School=1, Bachelor=2, Master=3, PhD=4
Gender	gender_Male	1 when the employee is male, otherwise 0
Marital status	marital_status_Married / Single	Separate 0/1 indicators
Department	department_HR / IT / Marketing / Operations / Sales	One column for each department
Employment type	employment_type_Full-Time / Part-Time	Numerical indicators
Age group	age_group_25-34 etc.	One indicator for each age group

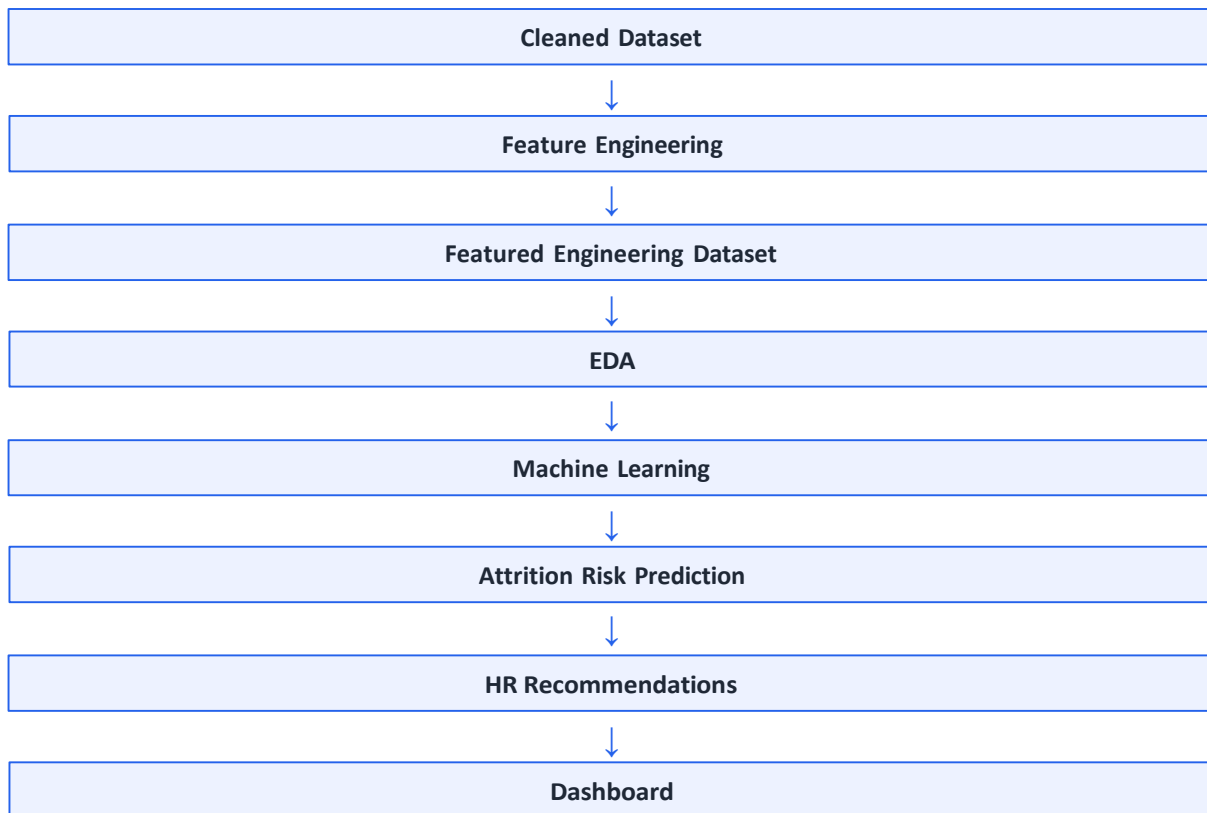
3.6 Dataset-Level Findings

Finding	Result
Total employees	15,000
Total columns	63
Employees who stayed (Attrition=0)	10,154 (67.69%)
Employees who left (Attrition=1)	4,846 (32.31%)
Missing values	0
Duplicate rows	0

3.7 Why This Dataset is Important

- EDA: Helps discover relationships between employee factors and attrition.
- Machine Learning: Provides numerical predictors for attrition prediction.
- Attrition Risk: Supports employee-level risk scoring.
- HR Recommendations: Helps identify areas where HR intervention may be useful.
- Dashboard: Provides fields for KPI cards, charts, filters and workforce dashboards.

3.8 Final Project Flow



4. Database Design & Data Loading

4.1 Overview

This section summarizes the database design and data-loading process, covering the relational schema design, the connection and security setup, the ETL script used to load cleaned and feature-engineered data into PostgreSQL (hosted on Supabase), the data-type handling required to match the schema, and the issues encountered and resolved during implementation.

Item	Detail
Database engine	PostgreSQL 15 (managed via Supabase)
Hosting	Supabase (Session Pooler connection, region: ap-northeast-1)
Input files	employee_attrition_cleaned_dataset.csv, Feature engineering.csv
Tables created	employees, employee_metrics, predictions
Rows loaded	15,000 rows into employees; 15,000 rows into employee_metrics
Load script	database/load_to_postgres.py
Schema file	database/schema.sql

4.2 Tech Stack

- PostgreSQL — relational database, hosted on Supabase's managed cloud platform
- pg8000 — Python driver used to connect to PostgreSQL and execute SQL
- pandas — reads the cleaned and feature-engineered CSV files into DataFrames
- python-dotenv — loads database credentials from a local .env file
- Supabase SQL Editor — used as a fallback for running schema DDL directly against the database

4.3 Database Schema Design

Three tables were designed to separate static employee attributes from computed metrics and model predictions, connected via foreign keys on employee_id. This separation keeps the raw employee record stable while allowing metrics and predictions to be recalculated or appended independently.

4.3.1 employees — core employee attributes

Column	Type	Notes
employee_id	INTEGER	Primary key
age	SMALLINT	
gender	VARCHAR(20)	
marital_status	VARCHAR(20)	
education_level	VARCHAR(30)	
department	VARCHAR(50)	Indexed
Column	Type	Notes

employment_type	VARCHAR(20)	
job_level	SMALLINT	
monthly_income	NUMERIC(12,2)	
years_at_company	SMALLINT	
years_in_current_role	SMALLINT	
created_at	TIMESTAMP	Default NOW()

4.3.2 employee_metrics — computed / derived metrics

Column	Type	Notes
metric_id	SERIAL	Primary key
employee_id	INTEGER	FK → employees, indexed
performance_rating	SMALLINT	
overall_satisfaction_index	NUMERIC(6,3)	
burnout_risk_score	NUMERIC(6,3)	
absence_rate_per_year	NUMERIC(6,3)	
hr_red_flag_count	SMALLINT	
is_new_hire	BOOLEAN	
career_stagnation_flag	BOOLEAN	
overtime_and_low_satisfaction_flag	BOOLEAN	
long_commute_flag	BOOLEAN	
high_performer_flag	BOOLEAN	
training_hours_last_year	SMALLINT	
attrition	BOOLEAN	Indexed
recorded_at	TIMESTAMP	Default NOW()

4.3.3 predictions — downstream ML output

Reserved for downstream Machine Learning output. Populated once the attrition model produces per-employee probability scores.

Column	Type	Notes
prediction_id	SERIAL	Primary key
employee_id	INTEGER	FK → employees, indexed
model_name	VARCHAR(50)	e.g. dataset1_attrition_rf_v1
model_version	VARCHAR(20)	
Column	Type	Notes

attrition_probability	NUMERIC(5,4)	0.0000 – 1.0000, indexed DESC
predicted_label	BOOLEAN	
predicted_at	TIMESTAMP	Default NOW()

4.3.4 Indexes

Five indexes were added to support common dashboard queries (filtering by department, attrition status, and joining across tables): `idx_employees_department`, `idx_metrics_attrition`, `idx_metrics_employee_id`, `idx_predictions_employee_id`, `idx_predictions_probability`.

4.4 Environment & Security Configuration

Database credentials are kept out of source control using a `.env` file (listed in `.gitignore`), loaded at runtime with `python-dotenv`: `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_SSL`. The connection uses Supabase's Session Pooler rather than a direct connection, since the pooler is IPv4-proxied and avoids connectivity issues on networks without native IPv6 support. SSL is enabled (`sslmode=require`) to encrypt traffic between the script and the database.

4.5 Data Loading Pipeline (ETL)

The load script (`database/load_to_postgres.py`) follows five sequential steps, each isolated in its own function for clarity and testability:

- **Connect** — establish a `pg8000` connection using credentials from `.env`, validating that all required variables are present before attempting to connect.
- **Apply schema** — check whether the target tables already exist; if not, execute `schema.sql` to create them.
- **Build tables** — read the cleaned and feature-engineered CSVs with `pandas`, standardize column names, and align them to the target column sets.
- **Transform** — cast flag columns stored as 0/1 integers in the CSV to true PostgreSQL booleans, since the schema defines them as `BOOLEAN`.
- **Load** — truncate existing rows to keep the load idempotent, then bulk-insert both tables in batches of 500 rows using `pg8000` for speed.

4.5.1 Idempotent Schema Application

Rather than always attempting to run the full DDL script (which can be slow through a connection pooler), the script first checks `information_schema` for an existing `employees` table and skips schema creation if it is already present.

4.5.2 Boolean Type Casting

The source CSVs store Yes/No flag fields as integer 0/1 values. Since the schema defines these columns as `BOOLEAN`, they are explicitly cast to Python `bool` before insertion to avoid a `pg8000.errors.DatatypeMismatch` error: `career_stagnation_flag`, `overtime_and_low_satisfaction_flag`, `long_commute_flag`, `high_performer_flag`, `attrition`.

4.5.3 Batched Bulk Insert

Both tables are truncated at the start of every run (RESTART IDENTITY CASCADE) so re-running the script is always safe and produces a clean, duplicate-free load. Rows are inserted in batches of 500 using `execute_values`, which is significantly faster than row-by-row inserts.

4.6 Execution & Verification

The final run of the load script completed successfully end-to-end: connection established, existing tables detected and schema creation skipped, 15,000 rows loaded into `employees`, 15,000 rows loaded into `employee_metrics`, data committed and connection closed. Row counts were independently verified in Supabase's Table Editor, confirming 15,000 populated rows in both tables, matching the source CSV row counts exactly.

4.7 Issues Encountered & Resolutions

Issue	Cause	Resolution
password authentication failed (28P01)	Database password not yet reset in Supabase, and/or a percent-encoded password (%40) was stored instead of the raw password	Reset the password in Supabase and stored the raw, unencoded value in <code>DB_PASSWORD</code>
Missing from <code>.env</code> : all variables	<code>.env</code> file was not being located at runtime	Loaded <code>.env</code> via an explicit absolute path (<code>BASE_DIR / ".env"</code>) instead of relying on the working directory
QueryCanceled: statement timeout	Schema DDL executed through the pooled connection exceeded the pooler's statement timeout	Applied <code>schema.sql</code> once directly in the Supabase SQL Editor, then added a table-existence check to skip DDL on later runs
Script appeared to hang	Bulk insert sent all 15,000 rows in a single statement with no progress feedback	Switched to batched inserts (<code>execute_values</code> , <code>page_size=500</code>) with progress print statements
DatatypeMismatch: boolean vs integer	Source CSV stores flag columns as 0/1 integers; schema defines them as <code>BOOLEAN</code>	Explicitly cast the five flag columns to Python bool with <code>.astype(bool)</code>
Verbose SQLAlchemy connection errors	SQLAlchemy's engine/pool/dialect abstraction produced long stack traces for simple issues	Replaced SQLAlchemy with direct <code>pg8000</code> connections and cursors

5. Exploratory Data Analysis (EDA)

5.1 Overall Attrition

The overall attrition rate is 32.3% roughly one in every three employees has left the organization. This is treated as the headline retention concern for the rest of the analysis.

5.2 Work – Arrangement Factors vs. Attrition

- Overtime: employees who work overtime have a 47.62% attrition rate vs. 25.02% for those who do not — a ~22.6-point gap, the single largest work-arrangement driver.
- Remote work: almost no effect (33.43% remote vs. 31.48% non-remote) — not a meaningful driver on its own.
- Business travel: frequent travellers leave at 41.84%, well above rare (30.44%) or never (30.21%) travellers.
- Employment type: contract employees are at 43.9%, noticeably higher than full-time (30.7%) or part-time (31.1%) staff.

5.3 Rating-Scale Factors vs. Attrition

- Job satisfaction: levels 1 and 2 show 50.55% and 52.26% attrition respectively — roughly double the ~25% rate at levels 3–5.
- Work-life balance: the lowest rating shows 48.85% attrition vs. ~27% for ratings 3–5, the same pattern as job satisfaction.
- Job level: attrition is highest among the most junior employees and decreases steadily with seniority, consistent with the tenure and promotion findings.

5.4 Attrition by Department and Job Role

- By department: HR has the highest attrition (33.31%), closely followed by Operations (33.24%); Marketing is lowest (31.42%). The spread is small, so attrition looks organisation-wide rather than one department's problem.
- By job role: Quality Analyst (35.05%), Training Specialist (34.67%), and Operations Executive (34.54%) are highest; Logistics Coordinator (29.60%) is lowest. The role spread is wider than the department spread, making role-level retention initiatives more promising.

5.5 Tenure, Pay, and Risk Score vs. Attrition

- Tenure: employees who left had a shorter median tenure (2 years vs. 4 years for stayers; means 3.59 vs. 4.92 years).
- Income: leavers earned less on average (8,045) than stayers (8,997) — a gap of about ₹952/month.
- Salary hike %: leavers received only a slightly lower average raise (14.51%) than stayers (15.11%) — about 0.6 points, suggesting recent raise size has little bearing on attrition by itself.
- Raw dataset's built-in Attrition_Risk_Score shows the clearest separation (leavers 63.87 vs. stayers 37.65 on average). This strong association is why the column was flagged as a possible leakage feature and excluded from modelling.

5.6 Correlation Analysis

The raw dataset's built-in Attrition_Risk_Score correlates most strongly with Attrition ($r = 0.60$). Among genuinely independent raw features, Overtime (+0.23) and Job Satisfaction (−0.20) are strongest, followed by Stock Option, Work-Life Balance, Job Level, and Monthly Income (roughly −0.13 to −0.17). Training Hours, Team Size, and Number of Projects show almost no linear relationship with attrition (each below 0.02 in magnitude).

5.7 Outlier Check

Outliers were minimal across the four numeric columns checked in this step, ranging from 2.09% to 3.57% of records, and reflect plausible extreme values (for example, unusually long commutes) rather than data-entry errors. No capping or removal was applied at this stage.

5.8 EDA Executive Summary

Overall attrition stands at 32.3%. The strongest and most actionable drivers are lack of promotion, overtime, low job satisfaction, poor work-life balance, contract employment status, and frequent business travel. Attrition skews toward early-career employees (leavers average ~2 years tenure vs. ~4 years for stayers). Department-level differences are minor — this looks like an organisation-wide issue — but specific roles (Quality Analyst, Training Specialist, Operations Executive) show meaningfully higher risk and are reasonable starting points for targeted retention efforts. Compensation plays a moderate role: absolute income shows a real gap between leavers and stayers, while the size of a recent raise does not. Data quality is solid overall — no missing values, no duplicate records, and minimal outliers across the numeric columns reviewed. The dataset's own Attrition_Risk_Score is strongly associated with attrition but was excluded from the predictive model. The EDA notebook was run on the cleaned dataset before feature engineering. Its leakage check is therefore part of the modelling-quality gate, not a post-hoc model adjustment.

6 Machine Learning Models

6.1 Dataset Used

Property	Details
Source	Kaggle — Employee Attrition & HR Analytics 2026
File	Machine Learning/Featured Engineering.csv
Total Records	15,000 employees
Original Columns	32 columns
After Feature Engineering	63 columns
Target Variables	Attrition (Model 1), Skill Gap (Model 2)

6.2 Feature Engineering Recap

The original 32-column dataset was expanded with 31 additional engineered columns (63 total). Key features feeding the models include:

Engineered Feature	Description
burnout_risk_score	Combines overtime, absence rate, and workload intensity
overall_satisfaction_index	Average of job, environment, and relationship satisfaction
career_stagnation_flag	Flags employees stuck in the same role for too long
commute_burden_index	Combines commute time and distance from home
income_percentile_in_dept	Employee income rank within their department

6.3 Data Leakage Handling

Data leakage occurs when a feature directly reveals the target variable, causing the model artificially high accuracy. The raw Kaggle dataset includes a pre-built Attrition_Risk_Score next to the Attrition target. EDA flagged this as a leakage risk, so it was deliberately excluded from Featured Engineering.csv and is not used as a model input. The attrition_risk_score later written to outputs/attrition_predictions.csv is a different, freshly computed Random Forest predict_proba() value.

6.4 Train-Test Split

The 15,000-record dataset was split 80/20 using a stratified split (stratify=y) to ensure both sets keep the same proportion of positive/negative target values, preventing biased evaluation.

Split	Records	Purpose
Training Set (80%)	12,000 records	Model learns patterns from this data
Testing Set (20%)	3,000 records	Unseen data used to evaluate real accuracy

6.5 Model 1 — Attrition Prediction

Objective: predict whether an employee will leave the company (1) or stay (0). Algorithm: Random Forest Classifier.

Reason	Explanation
Handles mixed data	Works with both numerical and categorical features
Resistant to overfitting	Multiple trees reduce variance compared to a single decision tree
Feature importance	Provides importance scores — helps HR identify key attrition drivers
Best accuracy	Outperformed Logistic Regression (74.57%) with 75.53% accuracy

6.5.1 Performance Metrics

Metric	Class 0 (Stay)	Class 1 (Leave)	Overall
Accuracy	-	-	75.53%
Precision	76%	71%	75%
Recall	92%	40%	76%
F1-Score	84%	52%	73%
Support	2,031	969	3,000

6.5.2 Confusion Matrix

	Predicted: Stay (0)	Predicted: Leave (1)
Actual: Stay (0)	1,875 (True Negative)	156 (False Positive)
Actual: Leave (1)	578 (False Negative)	391 (True Positive)

6.5.3 Top 10 Features Driving Attrition

Rank	Feature	Importance
1	Job Satisfaction	8.21%
2	Overall Satisfaction Index	6.17%
3	Overtime	5.59%
4	Burnout Risk Score	4.20%
5	Commute Time (Minutes)	4.00%
6	Absence Rate Per Year	3.56%
7	Distance From Home	3.50%
8	Work Life Balance	3.42%
9	Manager Rating	3.41%
10	Salary Hike Per Year Tenure	2.95%

6.6 Model 2 — Skill Gap Prediction

Objective: train_skill_gap.py does not train against a ground truth HR skills assessment. It first creates a synthetic proxy label named skill_gap using a weighted rule(long tenure + low job level, below – department – median pay, no promotion in 5 years, low performance) plus random noise, thresholded at ≥ 3 . The five rule building columns are dropped before training.

LightGBM (Light Gradient Boosting Machine) was selected after comparing four algorithms. It uses gradient boosting with leaf-wise tree growth, making it faster and more accurate than alternatives for this dataset:

Algorithm	Accuracy
LightGBM (Selected)	79.80%
Random Forest	75.00%
XGBoost	76.00%
Logistic Regression	73.00%

6.6.1 Performance Metrics

Metric	Class 0 (No Gap)	Class 1 (Gap)	Overall
Accuracy	-	-	79.80%
Precision	84%	75%	80%
Recall	81%	78%	80%
F1-Score	82%	76%	80%
Support	1,739	1,261	3,000

6.6.2 Confusion Matrix

	Predicted: No Gap (0)	Predicted: Gap (1)
Actual: No Gap (0)	1,408 (True Negative)	331 (False Positive)
Actual: Gap (1)	275 (False Negative)	986 (True Positive)

6.6.3 Top 5 Features Driving Skill Gap

Rank	Feature
1	Income Per Job Level
2	Income Percentile in Department
3	Monthly Income
4	Performance Manager Gap
5	Commute Burden Index

6.6.4 Interpretation of the 79.80% Accuracy

The LightGBM model's 79.80% accuracy measures prediction of this synthetic proxy label, not a validated real – world skills assessment. The model is therefore best described as a proxy – risk predictor until genuine HR skills- assessment target is available.

6.7 Output Files

File	Location	Used By
attrition_model.pkl	models/	Backend API — load & serve predictions
skill_gap_model.pkl	models/	Backend API — load & serve predictions
attrition_predictions.csv	outputs/	Dashboard — visualizations
skill_gap_predictions.csv	outputs/	Dashboard — visualizations

6.8 Model Summary

Model	Algorithm	Accuracy	Script	PKL File
Attrition Prediction	Random Forest (200 trees)	75.53%	Machine Learning/train_attrition.py	attrition_model.pkl
Skill Gap Prediction	LightGBM (200 estimators)	79.80%	Machine Learning/train_skill_gap.py	skill_gap_model.pkl

6.9 How to Run

Install dependencies: pip install pandas scikit-learn lightgbm joblib

Run from repo root: python "Machine Learning/train_attrition.py" and python "Machine Learning/train_skill_gap.py"

6.10 HR Recommendations Based on Model Insights

Key Driver	Recommended Action
Low Job Satisfaction	Conduct regular satisfaction surveys and take corrective action
High Overtime	Monitor and limit overtime hours; enforce work-life balance policies
High Commute Time	Offer remote/hybrid work options for employees with long commutes
Low Training Hours	Invest in L&D programs; mandatory training for skill-gap employees
Low Manager Rating	Manager effectiveness programs and 360-degree feedback

7 AI Recommendation Engine

7.1 What is the AI Recommendation Engine?

The recommendation engine acts as the bridge between machine learning predictions and human resources. Instead of only predicting who might leave or have skill deficiencies, it evaluates those probabilities and generates targeted, actionable HR strategies.

Scenario	Prescribed Action	Why it helps
High Attrition Risk	Immediate 1-on-1 & comp review	Directly addresses critical flight risks before the employee officially resigns.
Moderate Attrition Risk	Stay interview & workload check	Proactively identifies and mitigates dissatisfaction early.
Significant Skill Gap	Mandatory technical bootcamp	Closes critical functional deficiencies quickly and structurally.
Minor Skill Gap	Senior mentorship	Encourages peer-to-peer learning and gradual improvement without heavy intervention.

7.2 Inputs and Outputs

Component	Format	Meaning
Attrition Predictions	CSV	Input containing ML probabilities for flight risk.
Skill Gap Predictions	CSV	Input containing predicted_skill_gap scores from the ML model.
Recommendation Output	JSON	Clean, structured array of tailored HR actions ready for the backend.

7.3 Steps Followed

Step	What was done	Simple explanation
Step 1	Load ML predictions	Read the attrition and skill gap CSV files generated in Step 8.
Step 2	Diagnostic check	Identify exact column names and verify data availability.
Step 3	Merge datasets	Inner-join the attrition and skill-gap prediction CSVs on the real employee_id column (both files already carry it end-to-end from the raw dataset).
Step 4	Validate coverage	Print a warning if the join produces fewer rows than either source file, to catch any employee_id mismatch early.
Step 5	Apply Attrition Rules	Trigger retention actions based on ML probability thresholds (>0.40, >0.70).

Step 6	Apply Skill Rules	Trigger training actions based on gap score thresholds (>0.30, >0.60).
Step 7	Format Output	Convert Python dictionaries to a structured JSON format.
Step 8	Export	Save as recommendations.json for API consumption.

7.4 Main Rule Triggers — Simple Explanation

Trigger	What it means	Recommended Action
Attrition Prob > 0.70	High risk of leaving	Critical flight risk protocol. Schedule immediate 1-on-1.
Attrition Prob > 0.40	Moderate risk of leaving	Conduct a stay interview and assess current workload.
Skill Gap > 0.60	High deficiency in required skills	Significant gap. Enroll in mandatory technical upskilling bootcamp.
Skill Gap > 0.30	Minor deficiency in required skills	Minor gap. Assign a senior mentor for targeted development.

7.5 JSON Output Schema

To ensure seamless integration with the application backend, the rule logic is converted into a standardized JSON schema.

JSON Field	Example Output	Simple meaning
employee_id	1042	The unique identifier linking the recommendation to the employee.
risk_level	"High"	Categorized flight risk based on ML probability.
retention_action	"Conduct a stay interview."	Specific HR instruction to retain the employee.
training_action	"Assign a senior mentor."	Specific HR instruction to upskill the employee.

7.6 Engine-Level Findings

Metric	Result
Total employees processed	15,000 (the full dataset — attrition and skill-gap models are scored on every employee, not only the 3,000 held out for evaluation)
Total JSON fields per employee	4
Execution Type	Deterministic Rule-Based Logic
ID Matching Method	Inner join on the real employee_id column, with a printed warning if any employee_id fails to match

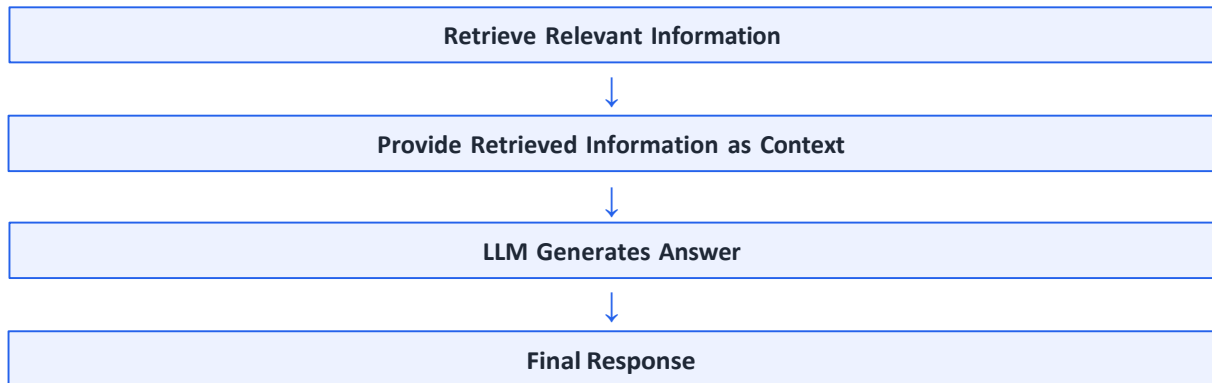
7.7 Why This Engine is Important

- Actionability: Turns raw ML numbers and probabilities into human-readable instructions.
- Automation: Eliminates manual HR reviews for standard risk/skill scenarios.
- Backend Integration: Provides the exact JSON schema required by the FastAPI backend.

8 RAG

8.1 Introduction to RAG

Retrieval-Augmented Generation (RAG) combines information retrieval with a Large Language Model (LLM). Instead of relying only on knowledge learned during training, RAG retrieves relevant information from a predefined knowledge base and supplies it as context to the LLM before generating an answer.



In this project, RAG is used to answer workforce-related questions using information maintained in the project's knowledge base.

8.2 What We Implemented

Component	Technology
Programming Language	Python
Document Format	Markdown
Document Processing	LangChain
Text Chunking	MarkdownHeaderTextSplitter
Retrieval Representation	scikit-learn TfidfVectorizer
Vector Database	FAISS IndexFlatIP (cosine search on normalised TF-IDF vectors)
Keyword Retrieval	BM25
Ranking Combination	Reciprocal Rank Fusion (RRF)
Reranking	TF-IDF cosine-similarity
Context Handling	Context Expansion
LLM	Groq-hosted model
Local LLM Runtime	Cloud API call (GROQ_API_KEY)

8.3 Knowledge Base

The current knowledge base contains five Markdown documents under `RAG/knowledge_base/documents/`: `career_growth.md`, `employee_retention.md`, `employee_skill_development.md`, `performance_management.md`, and `workforce_policy.md` — covering career growth, employee retention, skill development, performance management, and workforce management. The RAG prompt instructs the LLM to use only the supplied context and to state when information is not available in the knowledge base.

8.4 Document Loading and Structured Chunking

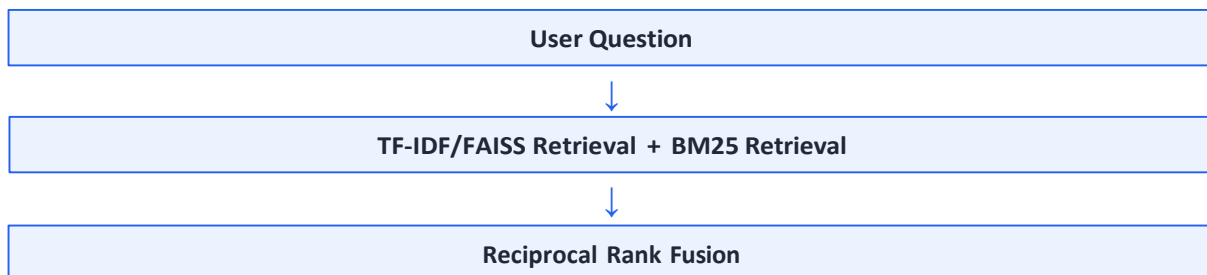
The Markdown files are split using `MarkdownHeaderTextSplitter`, which recognizes Markdown headers (`# Header 1`, `## Header 2`) to preserve meaningful document sections. Each chunk receives metadata: document name, source, parent section, section, chunk ID, and parent ID. The current five documents produce a total of 12 structured chunks.

8.5 TF-IDF Vectorisation and FAISS Vector Store

Each chunk is converted into a TF-IDF vector (unigrams and bigrams, sublinear term-frequency scaling) using scikit-learn's `TfidfVectorizer`, fitted once over all 12 knowledge-base chunks. The resulting vectors are L2-normalised and indexed in a FAISS `IndexFlatIP` index, so an inner-product search against a normalised query vector is equivalent to cosine similarity. The fitted vectorizer, FAISS index, and chunk metadata are persisted to `RAG/vectorstore/` (`tfidf_vectorizer.pkl`, `faiss_index.bin`, `chunks_metadata.pkl`) by `RAG/create_vectorstore.py`. This gives fast, dependency-light "semantic-ish" retrieval that captures lexical overlap and phrase structure, without downloading or running a neural embedding model.

8.6 Hybrid Retrieval

The retrieval stage combines TF-IDF/FAISS retrieval with BM25 keyword retrieval. TF-IDF/FAISS retrieval helps identify information based on lexical and phrase-level similarity, while BM25 is useful for matching important or exact keywords.



8.7 Reciprocal Rank Fusion

The results from TF-IDF/FAISS retrieval and BM25 retrieval are combined using Reciprocal Rank Fusion (RRF), producing a single ranked list before the reranking stage. RRF is implemented in `RAG/hybrid_retrieve.py` (`reciprocal_rank_fusion()`) using the standard $1/(k+\text{rank})$ scoring with $k=60$.

8.8 TF-IDF Cosine Reranking

The top-ranked candidates from RRF are re-scored in `RAG/reranker.py` by fitting a fresh TF-IDF vectorizer over the query plus the candidate chunks and ranking chunks by cosine similarity to the query vector.

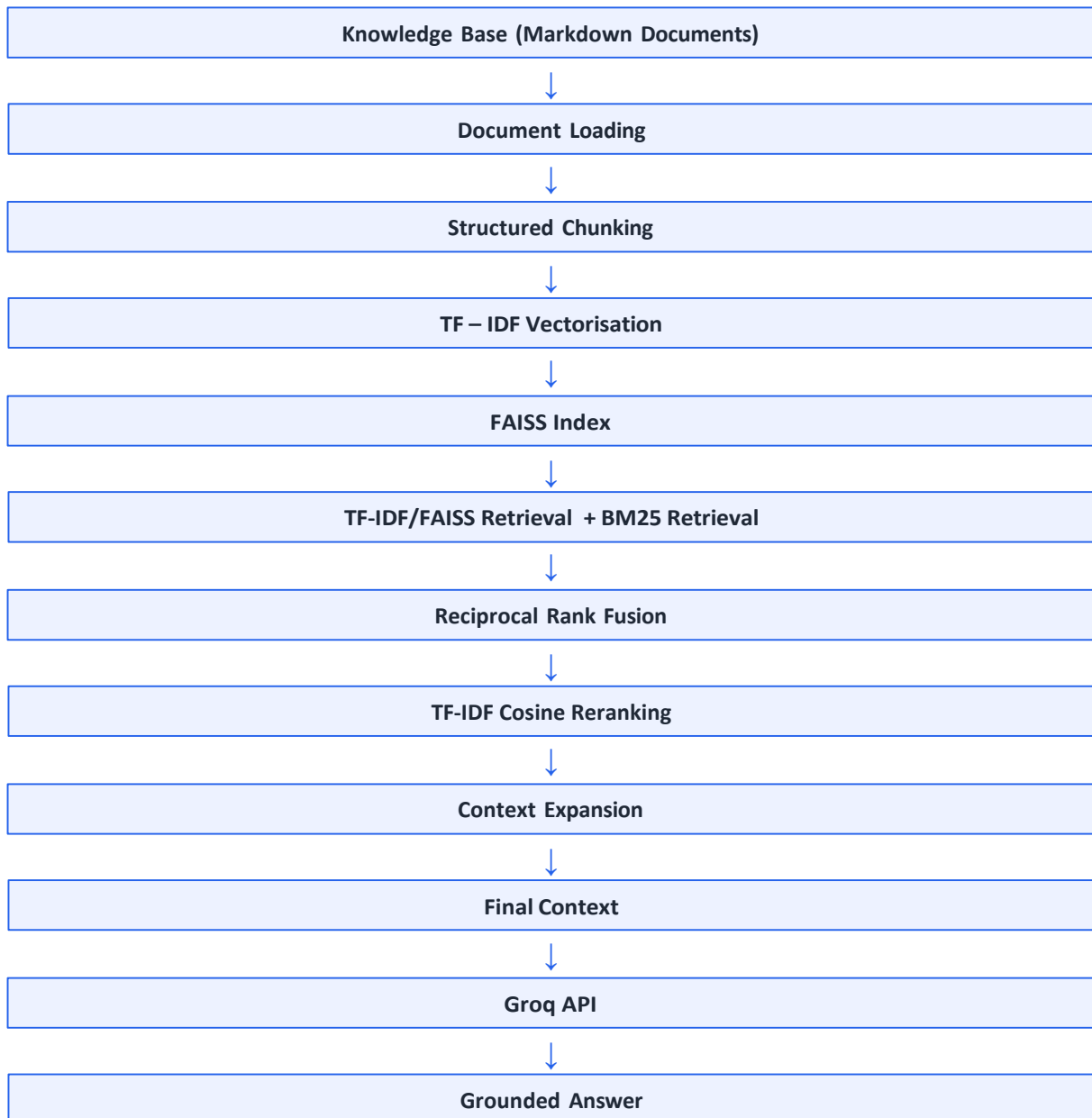
8.9 Context Expansion

After reranking, the selected information is expanded using the document and section relationships stored in chunk metadata (RAG/context_expander.py): it pulls in the neighbouring chunk(s) from the same source document (window_size=1) around each reranked chunk, using the chunk_id ordering stored in the metadata, so the LLM receives sufficient related information rather than a single isolated chunk.

8.10 Grounded Answer Generation

The final context is sent to the Groq API (RAG/llm.py, query_groq()) with a prompt instructing the model to answer only from the supplied context and say so when the answer isn't in the knowledge base. This requires a GROQ_API_KEY environment variable (free tier available at console.groq.com); if the key is missing or the API call fails, the function returns a fixed message telling the user to configure it, rather than falling back to a local mode.

8.11 Complete RAG Pipeline



8.12 Main RAG Files

File	Purpose
load_documents.py	Document loading
chunk_documents.py	Structured Markdown chunking and metadata
create_vectorstore.py	TF-IDF vectorisation and FAISS index creation
hybrid_retrieve.py	TF-IDF/FAISS retrieval, BM25 and RRF
reranker.py	TF-IDF cosine-similarity reranking
context_expander.py	Context expansion
llm.py	Groq API answer generation
main.py	End-to-end RAG pipeline

8.13 Testing Completed

- Python RAG packages were successfully imported.
- A GROQ_API_KEY was configured and a live call to the Groq API was verified to return a grounded answer.
- The TF-IDF vectorizer fitted successfully; the FAISS index was created successfully.
- All 12 structured chunks were embedded (vectorised).
- Retrieval was tested using the query "What are the guidelines for employee workload?" — the retrieval stage returned relevant workforce information, including workload as a relevant workforce indicator.

8.14 Running the RAG Pipeline

- Activate the virtual environment: `.\venv\Scripts\Activate.ps1`
- Set GROQ_API_KEY in your .env file (obtain a free key at console.groq.com)
- Rebuild the vector store: `python .\RAG\create_vectorstore.py`
- Run the complete RAG pipeline: `python .\RAG\main.py`

9 Frontend Design

9.1 Purpose

The dashboard is the single interface through which HR stakeholders consume the outputs of the data cleaning, feature engineering, machine learning, and recommendation stages described above. It turns model probabilities and engineered features into readable KPIs, charts, and per-employee profiles, without requiring any technical knowledge from the end user.

9.2 Technology Stack

Layer	Technology	Role
Server / routing	FastAPI	Serves HTML pages and computes page-level statistics from the dataset and trained models
Templating	Jinja2	Injects Python-computed values into HTML using <code>{{ }}</code> placeholders
Styling	Custom CSS (style.css)	A dark, glassmorphism-style corporate theme with CSS custom-property design tokens
Charting	Chart.js	Renders donut, bar, and line charts from JSON data embedded in each page
Data source	Featured Engineering.csv + attrition_model.pkl + skill_gap_model.pkl	Loaded once at startup; used to compute KPIs and per-employee records on every request

9.3 Design System

A consistent visual language is defined once, in CSS custom properties, and reused across every page:

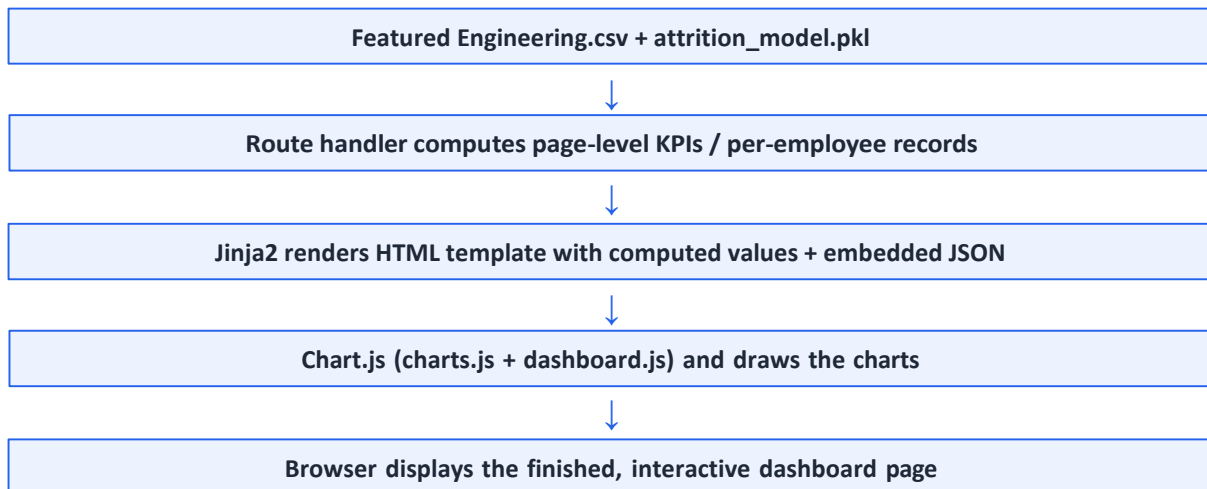
Token	Value	Used for
--bg-main	#0f172a (near-black navy)	Page background
--bg-card	#1e293b	Card backgrounds
--bg-sidebar	#0b1120	Fixed left navigation sidebar
--accent-primary	#38bdf8 (sky blue)	Highlights, links, primary buttons
--accent-success / warning / danger	#10b981 / #f59e0b / #ef4444	Risk badges and status indicators
--radius-sm / md / lg	8px / 12px / 16px	Rounded corners on cards and buttons
--shadow-card	Layered box-shadow	Card elevation

The layout uses a fixed 260px sidebar plus a fluid main content area, with a 12-column grid system (col-8, col-4, col-6, col-12) for arranging chart and stat cards responsively.

9.4 Page Inventory

Page	Route	Purpose
Dashboard	/dashboard	Executive summary: 5 KPI cards, 4 charts (department attrition, risk cohorts, skill readiness, satisfaction vs. turnover), and a critical flight-risk table
Employees	/employees	Searchable, filterable, paginated directory of all employees with risk and readiness badges
Employee Details	/employee/{id}	Individual profile: demographics, satisfaction scores, risk factors, and personalized retention/training recommendations
Attrition Analysis	/attrition	Deeper attrition diagnostics: overtime impact, promotion lag, commute risk, and top ML feature importances
Skill Gap Matrix	/skill_gap	Department-level skill-readiness breakdown and readiness-tier distribution
ML Simulator	/predictions	What-if form that calls the trained model live to score a hypothetical employee
AI Recommendations	/recommendations	Aggregate view of the recommendation engine's rule-based HR actions

9.5 Data Flow into the Dashboard



9.6 Chart Library Design

charts.js centralizes reusable chart builders (createDonutChart, createBarChart, createRadarChart) with a shared color palette and consistent tooltip styling, so every page's charts look and behave the same way. dashboard.js then calls these builders with page-specific data — for example, a donut chart for attrition-risk cohorts and a bar chart for department-level attrition rate — keeping chart configuration separate from chart usage.

9.7 Key UI Components

- Stat cards — headline KPI, an icon badge, and a supporting sub-label (e.g. Total Headcount, Attrition Rate, High Flight Risk, Avg Satisfaction, Skill Readiness).
- Chart cards — a title, subtitle, and a canvas-based Chart.js visualization inside a consistent card frame.
- Risk & readiness badges — color-coded pills (green/amber/red) that make an employee's status readable at a glance without reading raw numbers.
- Data tables — sortable, paginated employee tables with inline action buttons linking to full profiles.
- Sidebar navigation — a fixed left rail with active-state highlighting so users always know which page they are on.

9.8 Why These Design Choices

- Dark theme: reduces eye strain for HR staff reviewing dashboards for extended periods and gives status colors (green/amber/red) strong visual contrast.
- Card-based layout: breaks dense workforce data into digestible, scannable chunks rather than one long page.
- Consistent design tokens: changing one CSS variable (e.g. `--accent-primary`) restyles the entire application, making the dashboard easy to maintain and rebrand.
- Server-rendered HTML (Jinja2) rather than a client-side framework: keeps the stack simple, avoids a separate build pipeline, and lets the same Python code that runs the ML models directly produce the page data.

9.9 How to Run the Dashboard

- Install dependencies from `requirements.txt` into a virtual environment.
- Ensure the trained model files (`attrition_model.pkl`, `skill_gap_model.pkl`) are the real binaries, not Git LFS pointer files.
- From the project root: `cd Frontend`, then run `python app.py` (or `uvicorn app:app --reload --port 5000`).
- Open `http://127.0.0.1:5000` in a browser. The AI chat and three analytics endpoints additionally require the separate FastAPI backend and PostgreSQL database; all core dashboard pages work without them. The web dashboard's Attrition badge and Skill Readiness badge are rule – based dashboard calculations, not direct live model inference.

10 Backend API

10.1 Purpose

The backend is a second, separate FastAPI service (backend/main.py) from the one that serves the dashboard pages (Frontend/app.py). Where Frontend/app.py renders the HTML dashboard directly from the CSV and pickled models, the backend service is responsible for the two capabilities the dashboard cannot serve on its own: the AI chat assistant and live analytics queries against the PostgreSQL database. "the AI chat and two analytics endpoints additionally require the separate FastAPI backend and PostgreSQL database" is the direct evidence for this split.

10.2 Architecture

Component	Role
backend/main.py	FastAPI application exposing the chat and analytics endpoints; runs as a separate process from the Frontend dashboard app.
backend/database.py	Database access layer. Builds a SQLAlchemy engine with drivename="postgresql+pg8000" — the pure-Python pg8000 driver, chosen so the backend needs no compiled C extension at deploy time (see Section 4.2).
RAG/	Invoked by the chat endpoint: hybrid retrieval → reranking → context expansion → Groq API call, returning a grounded answer.
PostgreSQL database	Backing store for the analytics endpoints, populated by database/load_to_postgres.py

10.3 Endpoints

The backend exposes at minimum the following, confirmed indirectly by what test_all_endpoints.py checks and by reference to "the AI chat and two analytics endpoints":

Capability	Confirmed by	Detail
Health check	test_all_endpoints.py	Confirms the backend process is up and reachable before other checks run.
Chat	test_all_endpoints.py	Accepts a question, runs it through the RAG pipeline (hybrid retrieval → RRF → TF-IDF reranking → context expansion → Groq API), and returns a grounded answer. Requires GROQ_API_KEY.
Analytics (×3)	test_all_endpoints.py	Three endpoints returning aggregate workforce statistics computed from the PostgreSQL database, returns department-level workforce statistics from PostgreSQL, returns at risk employee information from the KPIs Frontend/app.py.

10.4 Configuration

The backend requires the following environment variables to start correctly:

Variable	Purpose
DB_HOST, DB_NAME, DB_USER, DB_PASSWORD, DB_PORT	PostgreSQL connection parameters consumed by backend/database.py's SQLAlchemy engine
GROQ_API_KEY	Required by the chat endpoint's call into the RAG pipeline's Groq API step . If missing, the RAG layer returns a fixed configuration-error message rather than failing silently.

10.5 Running the Backend

- Ensure the PostgreSQL database is running and reachable, and that database/load_to_postgres.py has been run at least once to populate it
- Set the DB_* variables and GROQ_API_KEY in your environment or .env file.
- Start the service using start_backend.ps1 (Windows/PowerShell), or run the equivalent uvicorn backend.main:app command directly.
- The backend is intended to be deployed separately from the Frontend dashboard app for the hosting target.

11 Power BI Dashboard

11.1 What it is

The project includes an interactive **Power BI dashboard** designed to provide a comprehensive view of workforce analytics, employee attrition, risk indicators, and employee development and well-being. The dashboard is available as an editable Power BI workbook named **projectdashboardfinal.pbix**.

The dashboard consists of three main pages: **Overview**, **Attrition & Risk**, and **Development & Well-being**.

11.2 Dashboard Overview

The Overview page provides an executive-level summary of the organization's workforce and key employee analytics. The page presents the following key performance indicators:

- **Attrition Rate** — overall percentage of employees who have left the organization.
- **Total Employees** — total employee population
- **Active Employees** — number of currently active employees.
- **High Risk Employees** — number of employees identified within the high-risk category.
- **Average Satisfaction** — average employee satisfaction score.
- **Attrition Count** — total number of employees who have left the organization.

The page also includes the following visualizations:

- **Attrition Rate by Tenure** — shows how employee attrition varies across different tenure levels.
- **Employee Risk Distribution** — displays the distribution of employees across Low, Medium, and High risk categories.
- **Attrition by Department** — compares attrition rates across different departments.
- **Executive Read-out** — summarizes important workforce observations and highlights key retention signals.

Interactive filters are available for **Department, Marital Status, Gender, Education Level, Employment Type, and Age**, allowing users to analyze the dashboard dynamically.

11.3 Attrition & Risk

The Attrition & Risk page focuses specifically on employee attrition patterns and workforce risk indicators. The page contains the following visualizations:

- **Attrition by Education Level** — compares employee attrition rates across different education categories.
- **Attrition by Job Level** — shows how attrition varies across job levels.
- **Attrition by Employment Type** — compares attrition among Contract, Part-Time, and Full-Time employees.
- **Average Satisfaction and Attrition Rate by Department** — examines the relationship between average satisfaction and departmental attrition.
- **Average Burnout Risk by Department** — compares burnout-risk levels across departments.
- **Headcount by Age Group** — shows employee distribution across different age groups.

These visuals help identify employee segments and organizational characteristics associated with higher or lower attrition.

11.4 Development & Well-being

The Development & Well-being page focuses on employee career development, training, performance, attendance, and employee-level risk indicators.

The page includes the following visuals:

- **Career Stagnation** — displays the number of employees experiencing career stagnation, defined in the dashboard as no promotion or role change within the specified period.
- **Average Training Hours per Employee** — compares average training hours across departments.
- **Average Performance Rating by Department** — compares average employee performance ratings across departments.
- **Average Absence Rate by Department** — compares employee absence levels across departments.
- **Flight-Risk Watchlist** — provides employee-level information for employees identified with multiple HR risk indicators.

This page provides HR stakeholders with a focused view of employee development and workforce well-being while also highlighting employees requiring closer attention.

11.5 Interactive Dashboard Features

The Power BI dashboard provides interactive filtering and navigation capabilities. Users can filter the dashboard based on:

- **Department**
- **Marital Status**
- **Gender**
- **Education Level**
- **Employment Type**
- **Age**

The navigation menu allows users to move between the three analytical pages:

Overview → Attrition & Risk → Development & Well-being

The interactive design enables users to explore workforce metrics from an organization-wide perspective and then drill into specific areas such as attrition, employee risk, development, performance, and attendance.

11.6 Power BI Deliverable

The final Power BI dashboard is provided as: **File Name: projectdashboardfinal.pbix**

The .pbix file contains the complete interactive Power BI report, including the Overview, Attrition & Risk, and Development & Well-being pages, together with their KPIs, visualizations, filters, and navigation elements.

The Power BI dashboard serves as an additional analytical and visualization layer alongside the project's web-based dashboard, machine-learning models, recommendation engine, and RAG-based AI assistant.

12 Testing & Deployment

12.1 Backend API Testing

`test_all_endpoints.py` exercises the backend FastAPI service (Section 8) end-to-end, running as a standalone script against a live instance of the backend. It checks four capabilities:

Check	What it confirms
Health Check	The backend process is running and reachable before any functional checks are attempted.
Analytics	The two analytics endpoints return valid responses when queried against the live PostgreSQL database.
Predictions	The attrition and skill-gap models can be invoked and return a scored result for sample input — the same underlying models used by the Frontend's ML Simulator page
Chat	A sample question sent to the chat endpoint returns a grounded answer, exercising the full RAG pipeline (Section 7) including the live Groq API call.

Run it with the backend service already started (Section 8.5) and reachable at its configured host/port:
`python test_all_endpoints.py`

12.2 Frontend Page Testing

`test_frontend_pages.py` checks that every Jinja2-rendered page in the dashboard returns successfully — an HTTP 200 response with non-empty rendered HTML — for each of the seven routes:

- Dashboard (`/dashboard`)
- Employees (`/employees`)
- Employee Details (`/employee/{id}`)
- Attrition Analysis (`/attrition`)
- Skill Gap Matrix (`/skill_gap`)
- ML Simulator (`/predictions`)
- AI Recommendations (`/recommendations`)

This confirms the seven-page inventory in Section 9.4 is accurate and that each page renders without a server error against the current dataset and model files.

`python test_frontend_pages.py`

12.3 Deployment

`start_backend.ps1`, a PowerShell script at the repo root, starts the backend FastAPI service (Section 8) — implying the primary local/deployment workflow targets Windows. `RAG/hybrid_retrieve.py`'s own code comments reference Render's free-tier size limits as the reason the RAG stack was rebuilt around TF-IDF/FAISS instead of sentence-transformer embeddings (Section 7.5), which is the strongest evidence that Render is the intended hosting platform for at least the backend service.

Required environment variables for deployment (consolidated from Sections 4.4, 7.10, and 8.4):

Variable	Used By
DB_HOST, DB_NAME, DB_USER, DB_PASSWORD, DB_PORT	backend/database.py (PostgreSQL connection) — Section 4.4
GROQ_API_KEY	RAG/llm.py's Groq API call, invoked via the backend's chat endpoint — Section 7.10

Two components need separate deployment:

- The Frontend dashboard (Frontend/app.py) — self-contained, needs only the CSV and .pkl model files (Section 9.5).
- The backend service (backend/main.py) — needs the PostgreSQL database and the environment variables above, started via start_backend.ps1 or its uvicorn equivalent.

13 Conclusion

This project delivers a complete, reproducible pipeline for workforce analytics: starting from a raw 15,000-employee HR dataset, the team cleaned and validated the data, engineered 31 additional predictive features, persisted the results in a properly indexed PostgreSQL schema, trained and evaluated two machine learning models (75.53% accuracy for attrition prediction, 79.80% for skill-gap prediction), converted model outputs into actionable HR recommendations through a rule-based engine, built a grounded RAG assistant for workforce-policy questions, and presented everything through a cohesive, dark-themed, chart-driven web dashboard.

Each stage was documented independently by the responsible team member, and this report consolidates that work into a single reference for academic submission and future maintenance.

13.1 Team Contributions

Stage	Owner
Data Cleaning	Santhosh, Satish
Feature Engineering	Thivya, Santhosh, Srishti
Machine Learning Models	Surendhar
Database Design & Loading	Srishti
AI Recommendation Engine	Gowtham
RAG Component	Satish, Anil
Frontend & Dashboard Design	Harshitha, Gowtham, Srishti
EDA	Anil, Harshitha
Backend	Srishti
Testing	Surendhar
Deployment	Thivya, Srishti